

ML Problem Solving

Multiclass Perceptron

Part 1: Feature Representation, Data Preprocessing & Geometry

Consider the following dataset $\mathcal{D}$, where each row represents a flattened 2×2 grayscale image followed by its ground-truth multiclass target label $y \in \{0, 2, 4\}$:

$$\mathcal{D} = \begin{bmatrix} 255 & 128 & 128 & 0 & 0 \\ 55 & 128 & 128 & 128 & 2 \\ 192 & 128 & 128 & 0 & 0 \\ 100 & 128 & 128 & 100 & 2 \\ 30 & 64 & 128 & 30 & 4 \\ 20 & 64 & 128 & 0 & 4 \end{bmatrix}$$

The input vector for sample k is given in row-major order:

$$\mathbf{x}^k = [b_{0,0}, b_{0,1}, b_{1,0}, b_{1,1}]^T$$

where $b_{r,c}$ denotes the pixel byte at row r , column c with intensity $b_{r,c} \in [0, 255]$.

Problem 1.1: Image Decoding & Visual Topology

1. For each of the 6 samples, draw the corresponding 2×2 grid representation. Annotate each pixel with its exact intensity value, and shade the cells proportionally (darker = lower value, lighter = higher value).
2. Categorize the visual patterns associated with each class ($y = 0$, $y = 2$, $y = 4$). What geometric feature (e.g., vertical edges, horizontal gradients, diagonal balance) intuitively distinguishes class 0 from classes 2 and 4?

Problem 1.2: Feature Normalization & Conditioning

1. Normalize the feature matrix $\mathbf{X} \in \mathbb{R}^{6 \times 4}$ into the range $[0, 1]$ using standard Min-Max scaling:

$$x_{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Assume the theoretical bounds of pixel data ($x_{min} = 0, x_{max} = 255$). Write out the exact fractional and decimal (to 3 decimal places) normalized matrix $\mathbf{X}_{norm}$.

Problem 1.3: Label Representations & One-Hot Encoding

The target labels are currently drawn from $\mathcal{Y} = \{0, 2, 4\}$. 1. Construct the one-hot encoded matrix $\mathbf{Y}_{OHE} \in \mathbb{R}^{6 \times 3}$, where the columns correspond to classes $c_0 = 0$, $c_1 = 2$, and $c_2 = 4$. 2. In a binary One-vs-Rest (OvR) framework for class 0, specify the binary target vector $\mathbf{y}^{(class=0)} \in \{0, 1\}^6$. 3. Contrast $\{0, 1\}$ binary target encoding with $\{-1, +1\}$ bipolar encoding. How does the weight update formula change between the two conventions?

Part 2: Step-by-Step Manual Trace of Perceptron Learning

Let the activation function of the Perceptron be the Heaviside step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0 \\ 0 & \text{if } z < 0 \end{cases}$$

where $z = \mathbf{w}^T \mathbf{x} + b = \sum_{i=1}^4 w_i x_i + b$.

The update rules with learning rate $\eta = 1.0$ are:

$$\Delta \mathbf{w} = \eta (y^k - \hat{y}^k) \mathbf{x}^k$$

$$\Delta b = \eta (y^k - \hat{y}^k)$$

where $\hat{y}^k = f(\mathbf{w}^T \mathbf{x}^k + b)$, and $y^k \in \{0, 1\}$ is the binary target.

Problem 2.1: Manual Pass over Epoch 1 (One-vs-Rest for Class 0)

Using the normalized dataset $\mathbf{X}_{norm}$ and target vector $\mathbf{y}^{(class=0)}$: - Initialize $\mathbf{w}_0 = [0, 0, 0, 0]^T$ and $b_0 = 0$. - Step through all 6 samples in order ($k = 1, 2, 3, 4, 5, 6$). - For each sample, tabulate: 1. Current weights $\mathbf{w}$ and bias b . 2. Input vector $\mathbf{x}^k$. 3. Pre-activation $z^k = \mathbf{w}^T \mathbf{x}^k + b$. 4. Predicted output $\hat{y}^k = f(z^k)$. 5. Target label y^k . 6. Prediction error $e^k = y^k - \hat{y}^k$. 7. Weight update $\Delta \mathbf{w}$ and bias update Δb . 8. Resulting updated weights and bias.

Problem 2.2: Tracking Classification Accuracy & Loss Dynamics

1. Immediately after **each** individual sample update in Problem 2.1, evaluate the current model against the **entire** 6-sample dataset $\mathbf{X}_{norm}$. Record:
 - The predictions $\hat{y} \in \{0, 1\}^6$ across all 6 samples.
 - The number of misclassified samples.
 - The dataset accuracy: $\text{Acc} = \frac{\text{Number of Correct Predictions}}{6} \times 100\%$.
2. Plot (or sketch) the accuracy curve as a function of the update step ($t = 0, 1, 2, \dots, 6$). Does accuracy increase monotonically? Explain why or why not.

Problem 2.3: Second Epoch Trace

Continue training into Epoch 2 (samples $k = 1$ to 6). 1. Perform the updates until convergence or completion of Epoch 2. 2. Does the linear unit for Class 0 achieve 100% training accuracy? If so, at which step?

Part 3: True Multiclass Perceptron (Kesler's Construction / Argmax Rule)

Rather than training independent binary classifiers (One-vs-Rest), a true Multiclass Perceptron maintains a weight vector $\mathbf{w}_c$ and bias b_c for every class $c \in \{0, 2, 4\}$.

The predicted class is:

$$\hat{c} = \arg \max_{c \in \{0, 2, 4\}} (\mathbf{w}_c^T \mathbf{x} + b_c)$$

If the prediction is incorrect ($\hat{c} \neq y^k$):

$$\begin{aligned} \mathbf{w}_{y^k} &\leftarrow \mathbf{w}_{y^k} + \eta \mathbf{x}^k, & b_{y^k} &\leftarrow b_{y^k} + \eta \\ \mathbf{w}_{\hat{c}} &\leftarrow \mathbf{w}_{\hat{c}} - \eta \mathbf{x}^k, & b_{\hat{c}} &\leftarrow b_{\hat{c}} - \eta \end{aligned}$$

Weights for all other classes $c \notin \{y^k, \hat{c}\}$ remain unchanged.

Problem 3.1: Multiclass Weight Trace

- Assume $\eta = 1.0$.
- Initialize $\mathbf{w}_0 = \mathbf{w}_2 = \mathbf{w}_4 = [0, 0, 0, 0]^T$ and $b_0 = b_2 = b_4 = 0$.
- In the event of an exact tie in arg max, use the deterministic tie-breaking rule: choose the smallest numerical class label (min c).
- Execute a full epoch (samples 1 through 6). Show the state of all three weight vectors and biases after each sample.

Problem 3.2: Comparison of Multiclass Frameworks

Compare the **One-vs-Rest (OvR)** scheme with the **Multiclass Perceptron (Argmax)** scheme: 1. How do they handle regions of ambiguity (e.g., where two OvR classifiers both predict 1, or all predict 0)? 2. What are the computational complexity differences during training and inference for K classes and D features?

Part 4: Geometric Analysis & Decision Boundaries

Problem 4.1: Equation of the Separating Hyperplane

Using the final weights and bias obtained from the Class 0 OvR Perceptron in Part 2: 1. Write down the algebraic equation of the decision boundary:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

2. Since $\mathbf{x} \in \mathbb{R}^4$, direct 2D visualization requires projection. Compute the Principal Component Analysis (PCA) projection or select features (x_1, x_4) (top-left vs. bottom-right pixels). Show algebraically how the decision boundary projects onto the (x_1, x_4) plane when holding x_2, x_3 at their mean values. 3. Compute the Euclidean distance from each of the 6 data points to this separating hyperplane:

$$d(\mathbf{x}^k, \mathcal{H}) = \frac{|\mathbf{w}^T \mathbf{x}^k + b|}{\|\mathbf{w}\|_2}$$

Which sample is the closest to the boundary (the critical margin sample)?

Part 5: Implementation in Python with `scikit-learn` & Custom NumPy

Now transition to code. Implement your solution in a Python script or Jupyter Notebook.

Problem 5.1: Custom NumPy Perceptron from Scratch

Write an object-oriented Python implementation:

```
class MulticlassPerceptronScratch:
    def __init__(self, n_features, n_classes, eta=1.0, max_iter=100):
        # Your code here
        pass

    def fit(self, X, y):
        # Implement the Kesler Argmax multiclass learning rule
        pass

    def predict(self, X):
        # Return predicted class labels
        pass
```

- Test this class on dataset $\mathcal{D}$.
- Print the exact sequence of weight updates and verify that your manual calculations from Part 3 match your code output.

Problem 5.2: `scikit-learn` Pipeline & Comparison

Using `sklearn.linear_model.Perceptron`: 1. Encode targets using `LabelEncoder` and scale features using `MinMaxScaler`. 2. Instantiate `Perceptron` with parameters: - `penalty=None` - `alpha=0.0` - `fit_intercept=True` - `max_iter=1000` - `tol=1e-3` - `shuffle=False` (to observe deterministic sample ordering) - `eta0=1.0` 3. Train the model on $\mathcal{D}$. 4. Inspect and display `clf.coef_` and `clf.intercept_`. 5. Explain how `scikit-learn` implements multiclass classification internally for `Perceptron` (does it use OvR or Argmax?).

Problem 5.3: Synthetic Scaled Experiment & Non-Separability

To evaluate convergence limits: 1. Generate a synthetic dataset of 300 samples with 3 classes using `sklearn.datasets.make_blobs` or `make_classification`:

```
from sklearn.datasets import make_classification
X_syn, y_syn = make_classification(
    n_samples=300, n_features=2, n_redundant=0,
    n_informative=2, n_clusters_per_class=1,
    n_classes=3, class_sep=1.5, random_state=42
)
```

2. Split into 70% train and 30% test sets.
3. Fit `Perceptron(max_iter=1000, random_state=42)`.
4. Plot the decision surfaces using `matplotlib` and `DecisionBoundaryDisplay` from `sklearn.inspection`.
5. Now set `class_sep=0.5` (creating significant class overlap / non-linear separability).
 - What happens to the convergence of the Perceptron algorithm?
 - How does the `early_stopping` and `tol` parameter prevent infinite loops in `sklearn`?
 - Report the Confusion Matrix, Macro-averaged Precision, Recall, and F1-score.